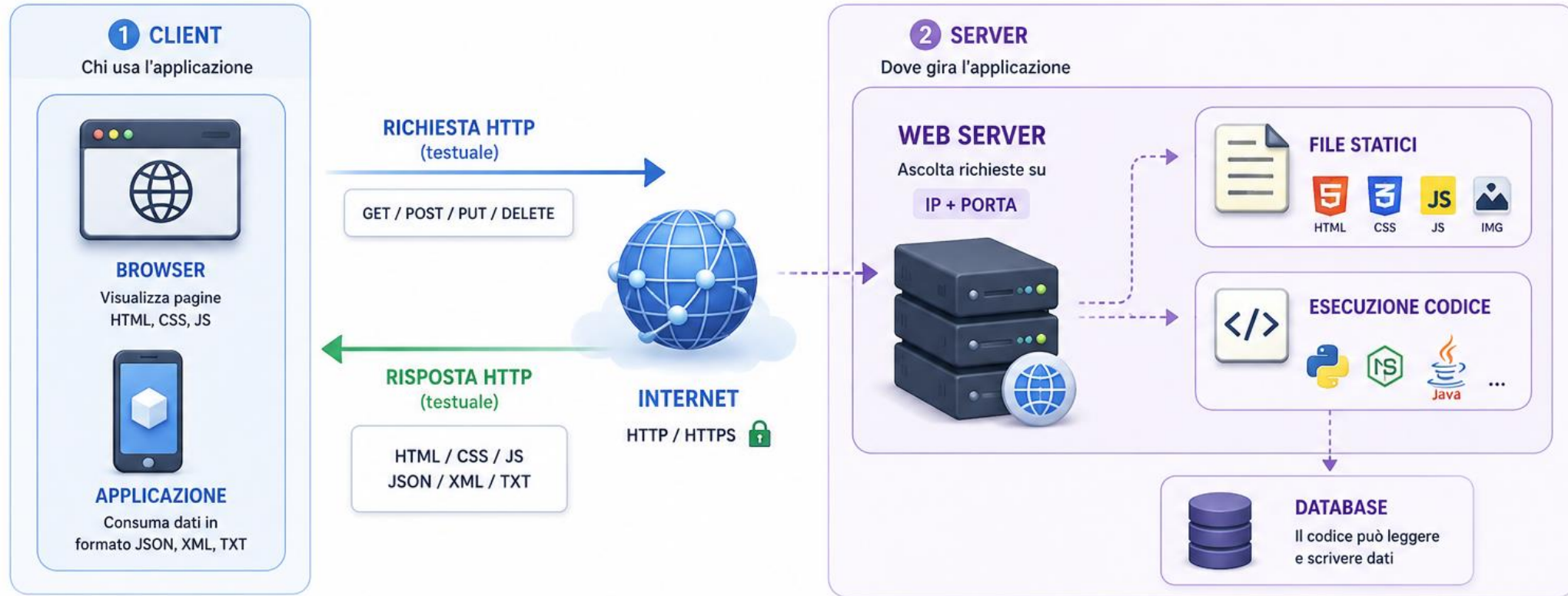


Architetture delle applicazioni web

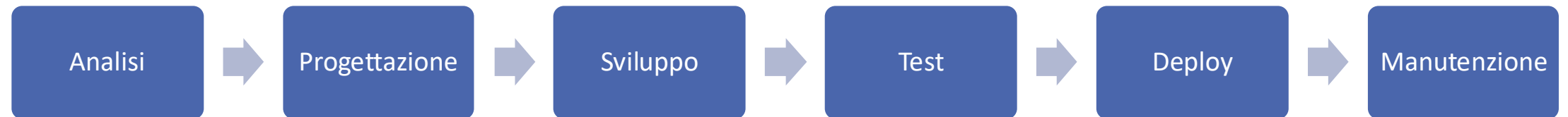
Architetture delle applicazioni web



Software Development Lifecycle

Software Development Life Cycle (SDLC)

- Con il termine Software Development Life Cycle si intende il processo utilizzato per pianificare le varie fasi dello sviluppo software, dall'analisi iniziale alla manutenzione finale.
- E' tipicamente composto da 6 fasi, ognuna delle quali prevede attività specifiche



SDCL - Analisi

- Si raccolgono tutti i requisiti che l'applicazione deve soddisfare
- I requisiti possono essere funzionali (legati alla logica applicativa) e non funzionali (legati alle performance, alla user experience, alla sicurezza...)
- I requisiti vanno raccolti tramite interviste, sondaggi e osservazioni sul campo. Devono essere coinvolte tutte le persone rilevanti (detti stakeholders)
- I requisiti vanno raccolti in un apposito documento, dentro al quale devono anche essere ordinati in base alla priorità/importanza/urgenze

SDCL - Progettazione

- Si progettano le varie parti architetturali che compongono l'applicazione:
 - Struttura del codice
 - Struttura dei database
 - User Interface
 - Caratteristiche dei server o delle piattaforme cloud
- Ogni tipologia di progettazione utilizza tecniche specifiche:
 - Struttura del codice → pattern architetturali, diagrammi UML (class, sequence, activity diagrams)
 - Struttura dei database → progettazione E/R
 - User Interface → User experience design
 - Caratteristiche dei server o delle piattaforme cloud → cloud engineering
- La progettazione deve essere coerente con le specifiche raccolte in fase di analisi

SDCL – Sviluppo e Test

- Nella fase di sviluppo si scrive il codice in modo conforme alle specifiche e alle scelte progettuali.
 - Si utilizzano sistemi di controllo di versione per tenere traccia delle modifiche e per permettere a team di collaborare
 - Si scrive il primo livello di test (unit testing)
- Nella fase di test si verifica che il codice sia conforme con tutti i requisiti (funzionali e non) e a tutte le scelte progettuali. Esistono quindi più livelli di test:
 - Unit e Integration testing: Per verificare l'assenza di bug nel codice
 - System testing: Per verificare la presenza di tutte le feature richieste e la corrispondenza con le specifiche
 - Performance testing
 - Usability testing
 - Security testing

SDCL – Deploy e Manutenzione

- Nella fase di Deploy il software viene pubblicato sull'infrastruttura di destinazione, anche detta infrastruttura di produzione
- La pubblicazione deve essere fatta in modo tale da:
 - Garantire la continuità operativa dell'applicazione
 - Garantire la possibilità di rollback in caso di problemi
- Nella fasi di manutenzione il software installato in produzione viene monitorato per evidenziare problemi e bug.
- Le anomalie riscontrate vengono gestite tramite appositi sistemi di bug tracking e vengono risolte in base alla priorità assegnata

Metodologie di sviluppo software

Le sei fasi viste in precedenza possono essere affrontate in più modi, dando origine a metodologie di sviluppo software differenti. Tra le principali ci sono:

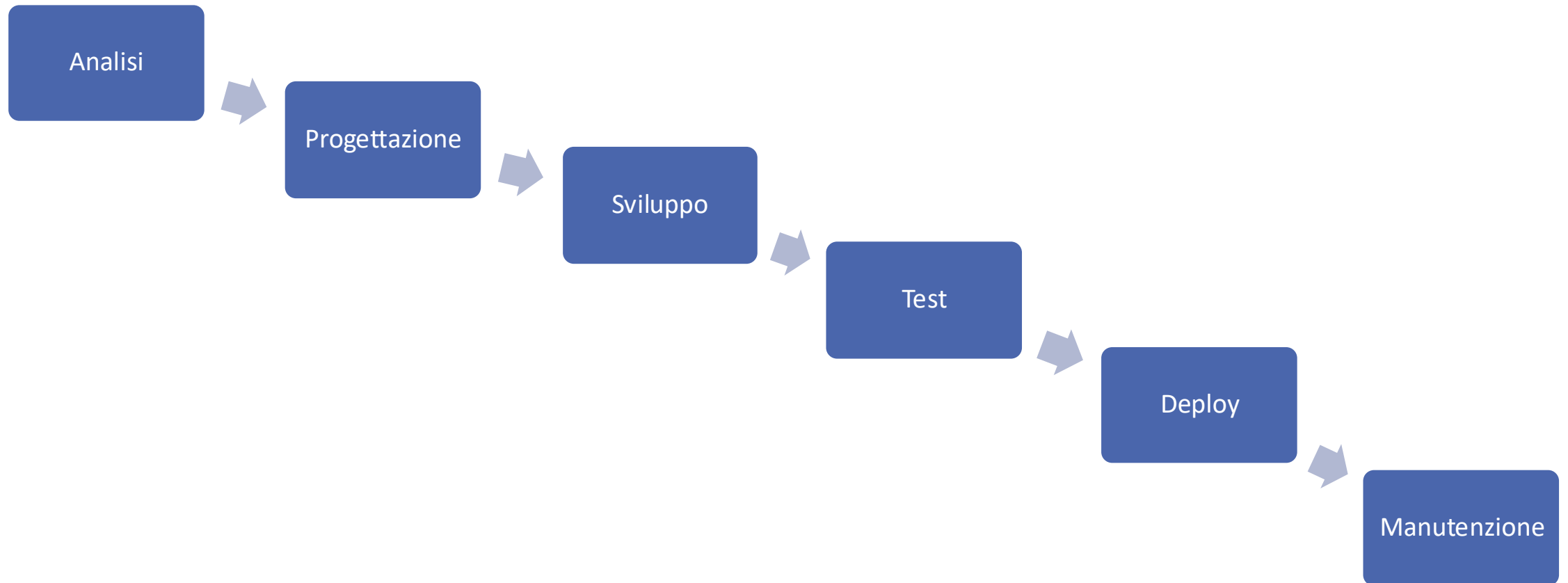
- Waterfall
- Metodologie Agili

Waterfall

In questa metodologia le sei fasi sono eseguite in stretto ordine sequenziale. Si può procedere ad una fase solo se la successiva è completamente terminata.

- L'approccio waterfall è adatto per grandi progetti, ben strutturati, con requisiti chiari e ben definiti.
- L'approccio waterfall non è adatto per progetti la cui struttura non è chiara e i cui requisiti possono cambiare in corso d'opera.

Waterfall

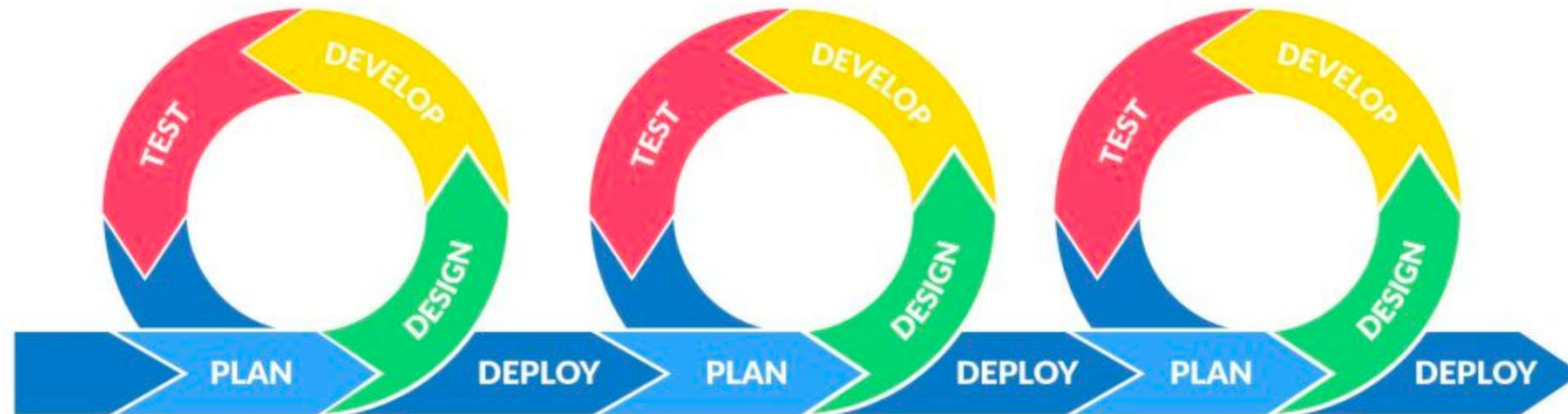


Agile

Le metodologie Agili (ne esiste più di una) sono volte alla flessibilità del processo di sviluppo.

Si caratterizzano dal rilascio di versioni intermedie del software, incomplete ma funzionanti, con frequenza mensile o bisettimanale.

Lo sviluppo procede pertanto con una sequenza di iterazioni dette sprint.



Agile Manifesto

- Il manifesto della programmazione agile (<https://agilemanifesto.org/>) si basa su 4 valori e 12 principi.
- Valori:
 - Individuals and interactions over processes and tools.
 - Working software over comprehensive documentation.
 - Customer collaboration over contract negotiation.
 - Responding to change over following a plan.

